

VLSI Design Internship- Task 2

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

Objective

In Task-1, you learned:

- What VLSI is and where it is used
- The complete VLSI design flow
- Digital logic fundamentals
- Logic gates and basic combinational circuits

In Task-2, you will take your **first practical step into VLSI implementation** by learning:

- Verilog Hardware Description Language (HDL)
- Register Transfer Level (RTL) design concepts
- Writing, simulating, and verifying combinational circuits using Verilog

This task marks the **transition from schematic-level thinking to code-based chip design**, which is how real VLSI engineers work.

Why This Task Matters (Industry Context)

In the semiconductor industry:

- All digital ICs are described using HDL (Verilog/SystemVerilog)
- Schematic design is rarely used beyond basic education
- RTL code is the foundation for synthesis, verification, and physical design

Companies expect VLSI interns to:

- Understand RTL concepts
- Write clean, synthesizable Verilog
- Verify functionality using testbenches

This task builds the **core skill required for all advanced VLSI roles**.

What You Will Build

You will:

- Write Verilog code for basic combinational circuits
- Create testbenches to verify functionality
- Simulate and analyze waveform outputs

Circuits to be implemented:

- Logic gates (AND, OR, NOT, NAND, NOR, XOR)
- Half Adder
- Full Adder

Concepts You Will Learn

Concept	Explanation
Verilog HDL	Hardware description language
RTL Design	Register Transfer Level modeling
Module	Basic building block in Verilog
Continuous Assignment	Combinational logic modeling
Testbench	Verification environment
Simulation	Functional validation

Tools & Software (Free / Open Source)

You may use **any one** of the following toolchains:

- Icarus Verilog + GTKWave (recommended)
- EDA Playground (online, no installation)
- ModelSim (Student version, optional)

No paid licenses are required.

Prerequisites

Before starting Task-2:

- Task-1 must be completed
- Basic understanding of logic gates and truth tables
- Familiarity with text editors and basic commands
- Willingness to learn coding concepts

Part A: Introduction to Verilog HDL

What is Verilog?

Verilog is a **hardware description language** used to describe:

- Digital circuits
- Logic behavior
- Hardware structure

Unlike programming languages:

- Verilog describes **hardware behavior**
- Code runs in parallel, not sequentially

Basic Structure of a Verilog Module

```
verilog

module module_name (input_ports, output_ports);
    // logic description
endmodule
```

Each module represents a **hardware block**.

Part B: Writing Your First Verilog Program

Step 1: Create a Verilog File

Create a file named:

```
logic_gates.v
```

Step 2: Implement Basic Logic Gates

Example: AND Gate

```
verilog

module and_gate (
    input A,
    input B,
    output Y
);
    assign Y = A & B;
endmodule
```

Implement Other Gates

Use the same structure to implement:

- OR gate (|)
- NOT gate (~)
- NAND gate (~&)
- NOR gate (~|)
- XOR gate (^)

Part C: Designing a Half Adder (RTL)

Half Adder Logic

Input A	Input B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Verilog Code: Half Adder

```
verilog

module half_adder (
    input A,
    input B,
    output Sum,
    output Carry
);
    assign Sum = A ^ B;
    assign Carry = A & B;
endmodule
```

Part D: Designing a Full Adder (RTL)

Full Adder Logic

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Verilog Code: Full Adder

```
verilog

module full_adder (
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);
    assign Sum = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

Part E: Writing a Testbench

What is a Testbench?

A testbench:

- Applies test inputs
 - Observes outputs
 - Verifies correctness
 - Is not synthesized into hardware
-

Sample Testbench for Half Adder

```
verilog

module tb_half_adder;

    reg A, B;
    wire Sum, Carry;

    half_adder uut (
        .A(A),
        .B(B),
        .Sum(Sum),
        .Carry(Carry)
    );

    initial begin
        A = 0; B = 0; #10;
        A = 0; B = 1; #10;
        A = 1; B = 0; #10;
        A = 1; B = 1; #10;
        $finish;
    end

endmodule
```

Part F: Simulation & Waveform Analysis

Simulation Steps (Icarus Verilog)

1. Compile:

```
nginx

iverilog logic_gates.v tb_half_adder.v
```

2. Run simulation:

```
csharp

vvp a.out
```

3. View waveforms (GTKWave):

```
lua

gtkwave dump.vcd
```

What to Verify

- Outputs match truth table
- No unknown (X) states
- Correct timing of signals

Expected Output

You should have:

- Working Verilog modules
- Passing testbench simulations
- Correct waveform results
- Understanding of RTL behavior

Key Learning Outcomes

After completing Task-2, you will be able to:

- Write synthesizable Verilog code
 - Design combinational circuits using RTL
 - Create and run testbenches
 - Simulate and debug logic
 - Prepare for synthesis-level tasks
-

Deliverables

Mandatory:

1. Verilog source files:
 - logic_gates.v
 - half_adder.v
 - full_adder.v
2. Testbench files
3. Simulation screenshots (waveforms)
4. Short PDF or document explaining:
 - Design logic
 - Simulation results

Optional:

- Combined top module
- Comments explaining each line

Free Learning Resources & Support Guide

(For Completing Task-2 Successfully)

To complete **Task-2 of the VLSI Design Internship**, interns should learn the required concepts using **freely available online learning resources**.

Instead of depending only on fixed video links, interns are encouraged to **search the topics listed below** on platforms such as:

- YouTube
- Google
- Educational websites
- AI learning tools (ChatGPT, Gemini, Perplexity)

Some links available online may change or be removed over time.

If a link is not working, simply **search the topic using the keywords provided below** to find updated tutorials.

The goal is to **understand concepts and apply them practically**, not just follow one specific resource.

Topics You Should Study

1. Verilog HDL Fundamentals

Learn the basics of **Verilog Hardware Description Language**, which is used to describe digital circuits.

Topics to search:

- What is Verilog HDL
- Verilog syntax basics
- Verilog module structure
- Inputs, outputs, and wires
- Continuous assignments (assign)
- Operators in Verilog

Example searches:

Verilog HDL basics for beginners

Verilog module structure explained

Continuous assignment in Verilog

2. RTL Design Concepts

RTL (Register Transfer Level) is the **core abstraction level used in digital chip design**.

Topics to study:

- What is RTL design
- RTL modeling in Verilog
- Combinational vs Sequential logic
- Hardware modeling concepts

Example searches:

RTL design explained in VLSI

What is RTL in Verilog

RTL modeling basics

3. Implementing Logic Gates in Verilog

In Task-2, you will implement basic digital logic gates using Verilog.

Gates to implement:

- AND gate
- OR gate
- NOT gate
- NAND gate
- NOR gate
- XOR gate

Example searches:

AND gate in Verilog

Logic gates implementation in Verilog

Verilog combinational logic examples

4. Designing Combinational Circuits

You will implement **basic combinational circuits** using Verilog.

Topics to study:

- Half Adder
- Full Adder
- Sum and Carry logic
- Boolean logic implementation

Example searches:

Half adder using Verilog

Full adder Verilog code

Combinational circuits in Verilog

5. Writing Testbenches (Very Important)

A **testbench** is used to test and verify Verilog modules.

Topics to search:

- What is a Verilog testbench
- Testbench structure
- Applying test inputs
- Observing outputs

Example searches:

Verilog testbench tutorial

How to write testbench in Verilog

Testbench example for half adder

6. Simulation and Waveform Analysis

Simulation verifies whether your design works correctly.

Topics to study:

- Verilog simulation flow
- Running simulation using Icarus Verilog
- Viewing waveforms using GTKWave
- Understanding signal timing

Example searches:

Icarus Verilog simulation tutorial

GTKWave waveform viewer tutorial

Verilog simulation step by step

Free Tools You Can Use

You can complete Task-2 using **free tools** such as:

Tool	Purpose
EDA Playground	Online Verilog editor and simulator
Icarus Verilog	Open-source Verilog simulator
GTKWave	Waveform viewer for simulation outputs

These tools allow you to **write, simulate, and analyze Verilog designs**.

Using AI Tools for Learning

AI tools can help you understand Verilog concepts faster.

Example prompts you can try:

Explain Verilog module structure with example

Write Verilog code for half adder

Generate a Verilog testbench for a full adder

Explain waveform output in Verilog simulation

Important Learning Advice

Do not rely on a single video or article.

Instead:

- Explore multiple tutorials
- Practice writing Verilog code
- Run simulations frequently
- Analyze waveform outputs carefully

This method helps you develop **real VLSI design skills**.

If You Face Any Difficulty

Try the following steps:

1. Search the topic again with different keywords
2. Refer to multiple tutorials or explanations
3. Use AI tools to clarify concepts
4. Experiment with Verilog code and simulations

Developing the ability to **research and solve problems independently** is a critical skill for VLSI engineers.

1. Free Tools & Online Platforms

EDA Playground (Recommended – No Installation)

Online Verilog/SystemVerilog editor with built-in simulators.

<https://www.edaplayground.com>

Icarus Verilog (Open Source Simulator)

Used for compiling and simulating Verilog.

<http://iverilog.icarus.com>

GTKWave (Waveform Viewer)

Used to view simulation waveforms.

<http://gtkwave.sourceforge.net>

2. Verilog HDL Basics (Beginner Friendly)

Verilog HDL Introduction (Clear and Structured)

<https://www.youtube.com/watch?v=KXcZ6XJ1p1M>

Verilog Tutorial for Beginners (Full Playlist)

<https://www.youtube.com/watch?v=0pFhQnPZ9hA>

Verilog HDL Basics – Modules and Operators

<https://www.youtube.com/watch?v=2yZ2y8R5Z6A>

3. RTL Design & Combinational Logic (Core for Task-2)

RTL Design Explained (What RTL Means in VLSI)

<https://www.youtube.com/watch?v=3sQp9YtJ5Qk>

Combinational Circuits in Verilog

<https://www.youtube.com/watch?v=6Z8X3FQkJ4M>

Continuous Assignment vs always Block

<https://www.youtube.com/watch?v=8tF6ZK3pY2A>

4. Logic Gates, Half Adder, Full Adder in Verilog

Logic Gates in Verilog (Hands-On)

<https://www.youtube.com/watch?v=9H8Z2M1FQJQ>

Half Adder in Verilog

<https://www.youtube.com/watch?v=7P2X8FJZk0Q>

Full Adder in Verilog (RTL Explanation)

<https://www.youtube.com/watch?v=1M7FZ9kJQpE>

5. Testbench Writing (Very Important)

What is a Testbench in Verilog

<https://www.youtube.com/watch?v=5Z9XkQ7FJ8E>

Writing a Simple Verilog Testbench

<https://www.youtube.com/watch?v=8KZ3F7JX9M2>

Testbench Example for Combinational Logic

<https://www.youtube.com/watch?v=2J9X8Z5FQkY>

6. Simulation & Waveform Analysis

Verilog Simulation Using Icarus Verilog

<https://www.youtube.com/watch?v=4Z8FJ2Qk9Xk>

GTKWave Tutorial for Beginners

<https://www.youtube.com/watch?v=6Q7XJZ8F9M2>

Understanding Waveforms (Timing & Signals)

<https://www.youtube.com/watch?v=3JZ8F2Qk9M5>

7. Free Academic Courses & Reading Material

NPTEL – Digital System Design (Free, High Quality)

<https://nptel.ac.in/courses/117/106/117106092>

NPTEL – VLSI Design Fundamentals

<https://nptel.ac.in/courses/108/107/108107129>

Verilog HDL Reference (Free PDF)

Search keyword:

“Verilog HDL reference guide PDF”

8. Free Documentation & References

Verilog Language Reference Manual (IEEE – Public Copies)

Search keyword:

“IEEE Verilog LRM PDF”

Icarus Verilog Documentation

<http://iverilog.icarus.com/docs>

GTKWave Documentation

<http://gtkwave.sourceforge.net/docs.html>

9. Practice & Examples (Very Useful)

EDA Playground Example Library

<https://www.edaplayground.com/examples>

Verilog Practice Search

https://www.youtube.com/results?search_query=verilog+combinational+circuits+examples

10. Community & Doubt Support (Free)

- EDA Playground Discussions
<https://groups.google.com/g/eda-playground>
 - Stack Overflow – Verilog Tag
<https://stackoverflow.com/questions/tagged/verilog>
 - Reddit VLSI Community
<https://www.reddit.com/r/VLSI/>
-

11. Recommended Learning Path for Task–2

Day 1

- Verilog syntax
- Modules and operators

Day 2

- Logic gates coding
- Half adder and full adder

Day 3

- Testbench writing
- Simulation basics

Day 4

- Waveform analysis using GTKWave

Day 5

- Code cleanup
 - Documentation and screenshots
-

12. Essentials Checklist for Task–2

You must clearly understand:

- Verilog module structure
- Continuous assignments
- Combinational logic modeling
- Testbench writing
- Simulation flow
- Waveform interpretatio

